Tilbe Yilmaz

October 20, 2026

Dr. Tong Zhang

ECSE 4320

Project 1: SIMD Advantage Profiling

Contents

Introduction 2
System Configuration 2
Run Procedure 2
Code Structure and Execution Flow 3
Hypothesis 3
Experiments and Results 3
Scalar Baseline and Auto Vectorization 3
Locality Sweep Across Cache Levels 4
Compiler Results 4
Data Type Comparison 6
Implication for SIMD 6
Alignment and Tail Handling 7
Stride and SIMD Speedup 8
Native Target Comparison 8
Roofline Interpretation 9
Ordered Dot as a Negative Example 9
Measured Cycles per Element 10
Validation and Reproducibility 10
Conclusion 11
Measurement and Validation Details 11
Limitations 11
AI Assistance 11
References 11

Introduction

This project examines when compiler-generated SIMD improves numerical performance and what limits that improvement. It compares scalar and vectorized versions of SAXPY, dot product, elementwise multiplication, and a three-point stencil while varying working set, data type, alignment, array length, and stride. Timing results are interpreted alongside compiler reports and disassembly. The repeat run provides the primary results, while the original run is used to assess reproducibility.

System Configuration

Item	Recorded configuration
Processor	AMD Ryzen 5 PRO 7530U; Zen 3; 6 cores and 12 logical processors. One benchmark thread was used [1].
Cache hierarchy	32 KiB L1 data and 512 KiB L2 per core; 16 MiB shared L3. The runner used --llc-mib 16.
Host and guest	Windows 11 x64; WSL 3.0.1; Linux 6.18.40.1-microsoft-standard-WSL2.
Compiler and ISA	GCC 11.4.0; guest AVX, AVX2, and FMA support. Generic loops use 128-bit arithmetic; native loops use 256-bit arithmetic.
Execution conditions	AC power and Best Power Efficiency in both runs. The repeat notes record no background activity.
Affinity and frequency	Pinned to vCPU 0, with SMT sibling 1. The 2 GHz clock is a roofline assumption; operating frequency was not measured.

Run Procedure

Before timing, each configuration is checked against a compensated long-double reference, warmed up, and calibrated toward a 100 ms batch. Seven batches are then recorded. Allocation, initialization, and reference checking are excluded from the timed interval. Each full run contains 1080 configurations and 7560 samples.

Runtime is the batch duration divided by its iteration count. Throughput is algorithmic work or useful bytes divided by runtime, and speedup is the matched scalar median divided by the vector median. Reported 95 percent intervals use 2000 unpaired bootstrap resamples. Cases and build blocks are randomized, although samples within each block remain consecutive.

Code Structure and Execution Flow

The code separates numerical work in src/, experiment automation in scripts/, and verification in tests. The same C++ kernels are compiled into four variants, allowing compiler-generated implementations to be compared without changing the source operations.

Component	Responsibility
src/kernels.hpp and kernels.cpp	Declare and implement the ordinary kernels. Templates specialize data type, stride, and alignment; dispatch selects the requested path.
src/strict_dot.cpp	Implements ordered summation in a separate translation unit so it can use -fno-fast-math.
src/benchmark.cpp	Manages buffers and inputs, validates outputs, warms up, calibrates batches, and records elapsed time, core cycles, and TSC ticks.
scripts/build.py and evidence.py	Compile the variants and retain commands, target macros, compiler diagnostics, disassembly, and an instruction index.
run_experiments.py and environment.py	Define the sweeps, record execution conditions, select affinity, randomize case order, and save raw.csv and commands.jsonl.
analyze.py and plot.py	Calculate statistics, matched speedups, traffic estimates, and the roofline model, then generate figures.
tests/ and run_all.sh	Check numerical behavior and analysis calculations, then coordinate building, experiments, analysis, and plotting.

The workflow runs from compiled kernels to raw samples, processed tables, and figures. Each results directory retains its own measurements and metadata, so analysis or plotting can be repeated without collecting new data.

Hypothesis

SIMD was expected to provide the largest gains when independent operations access contiguous data in cache. Gains were expected to shrink as data movement became more restrictive, while stride and ordered accumulation were expected to reduce the benefit. Float32 was expected to achieve greater throughput because twice as many values fit into a 256-bit vector.

Experiments and Results

Scalar Baseline and Auto Vectorization

Variant	Target flags	Vectorization	Baseline
scalar	-march=x86-64 -mtune=generic	Disabled	Reference
generic	-march=x86-64 -mtune=generic	Enabled	scalar
scalar_native	-march=native	Disabled	Reference
native	-march=native	Enabled	scalar_native

Ordinary kernels use -O3 -ffast-math -ffp-contract=off. Scalar variants disable loop and SLP vectorization. The harness is compiled separately at -O2 with LTO disabled, limiting caller optimization across repeated kernel calls. Matching the target makes native versus scalar_native the more direct SIMD comparison; native versus generic also changes ISA and tuning [2, 3].

Locality Sweep Across Cache Levels

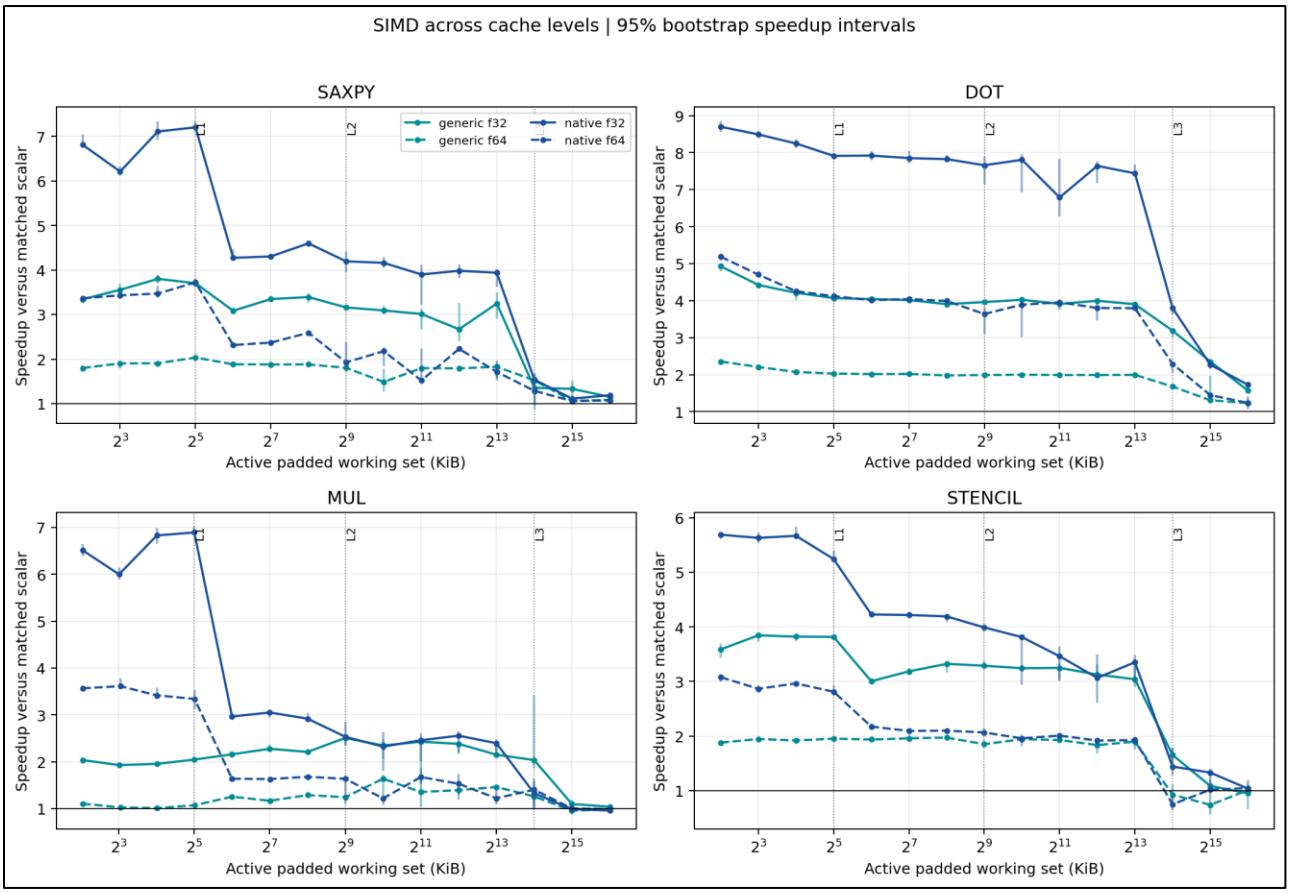


Figure 1. Speedup across working sets.

Native float32 peak speedups are 7.21x for SAXPY, 8.70x for dot product, 6.90x for multiplication, and 5.69x for stencil. These peaks occur at approximately 4 to 32 KiB footprints, where data can be supplied quickly from or near L1 cache. Native generally has a larger advantage than generic in this range, consistent with its wider packed loops.

The advantage decreases as working sets grow. Once footprints extend beyond shared L3 capacity, faster arithmetic often contributes less to total runtime because supplying and writing data becomes more expensive. Cache boundaries provide useful reference points, but speedup is a ratio of two changing runtimes and need not show a sharp transition at every boundary.

Compiler Results

GCC reports 32-byte vector loops at kernels.cpp lines 15, 19, 23, and 26. Native contiguous loops use packed multiplication and addition in YMM registers, processing eight float32 or four float64 values per iteration. Generic loops use 128-bit packed arithmetic. Scalar kernel objects contain no packed floating-point arithmetic, and no FMA instructions appear in the kernel objects.

For example, native float32 multiplication at execute<float, 1ul, true> offsets 0x590 to 0x5a6 contains vmovaps, vmulps, a packed store, a 32-byte pointer increment, and a backward branch. This connects SIMD instructions to the measured loop rather than relying on register names alone.

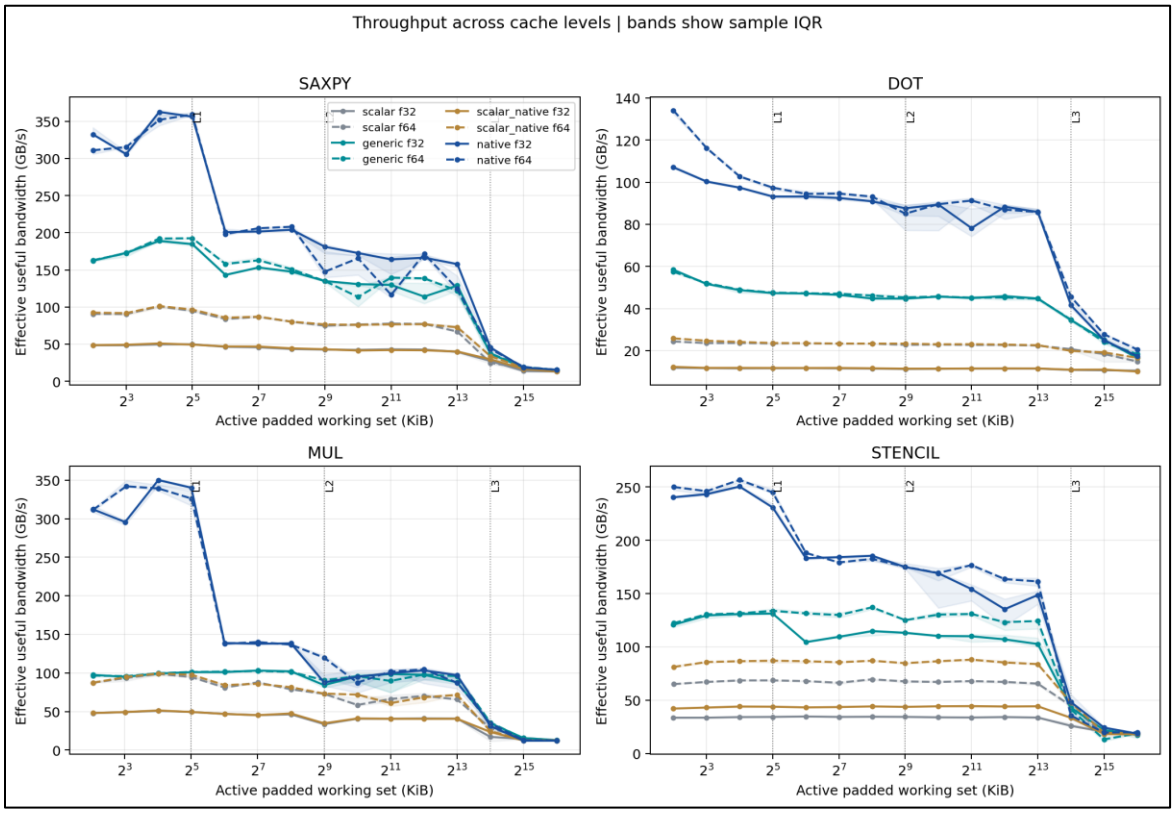


Figure 2. Useful throughput across working sets.

Cached SAXPY and multiplication reach about 363 and 350 GB/s of useful bandwidth. These values count source-level bytes processed through cache; they are not measurements of RAM bandwidth. At approximately 64 MiB, multiplication falls to about 12 GB/s and scalar/vector runtimes converge, consistent with data movement limiting the benefit of faster arithmetic.

Kernel at about 64 MiB	Scalar native ms	Native ms	Speedup and 95 percent interval
saxpy f32	7.636	6.424	1.189x (1.142 to 1.203)
saxpy f64	7.083	6.601	1.073x (1.030 to 1.192)
dot f32	6.654	3.851	1.728x (1.598 to 1.780)
dot f64	4.042	3.268	1.237x (1.074 to 1.404)
mul f32	5.349	5.580	0.959x (0.935 to 1.062)
mul f64	5.351	5.407	0.990x (0.912 to 1.013)
stencil f32	7.404	7.165	1.033x (0.985 to 1.192)
stencil f64	6.971	6.683	1.043x (1.005 to 1.070)

Multiplication has intervals containing 1x for both types, while float32 stencil also shows no clear advantage. Dot retains speedups of 1.728x for float32 and 1.237x for float64. Its packed lane sums reduce the accumulation dependency of the scalar baseline, showing that large-array performance can still depend on algorithm structure as well as memory access.

Data Type Comparison

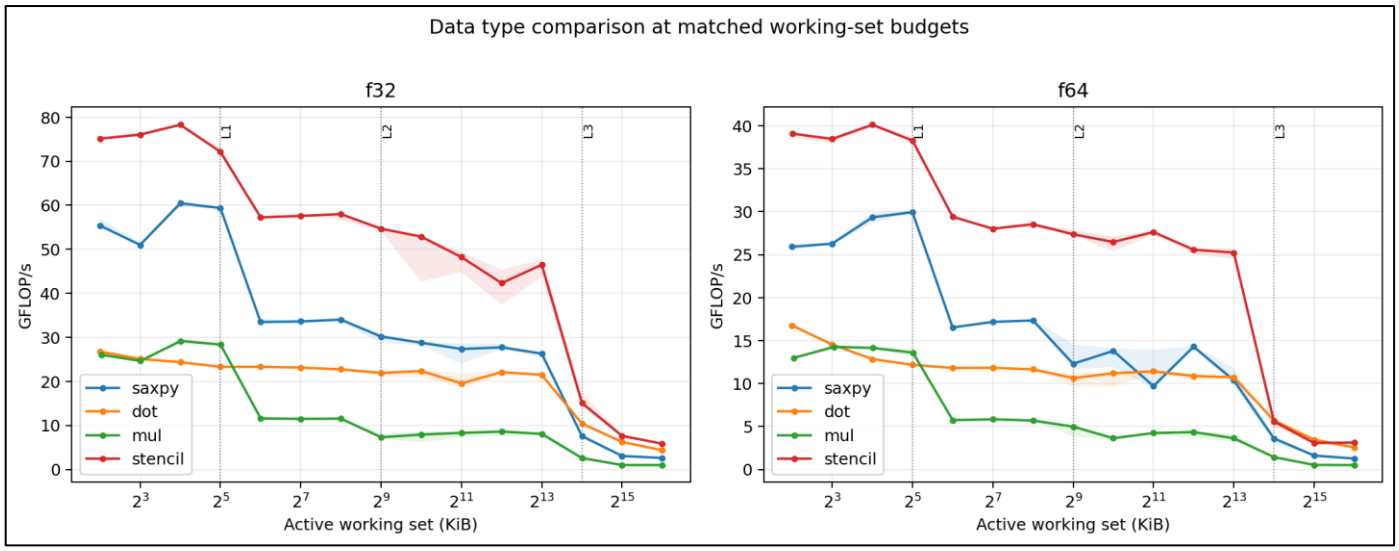


Figure 3. Arithmetic throughput by data type.

Kernel	Peak float32 GFLOP/s	Peak float64 GFLOP/s
SAXPY	60.44	29.94
Dot	26.78	16.77
Multiplication	29.19	14.26
Stencil	78.28	40.12

Float32 generally delivers greater arithmetic throughput because a 256-bit vector holds eight values rather than four, and each value requires half the bytes. The measured ratio varies because instruction mix, reduction dependencies, and cache behavior also affect execution. The plots approximately match working-set budgets, so the two types use different element counts.

GFLOP/s describes algorithmic work rather than outputs processed. SAXPY and dot count two FLOPs per output, multiplication counts one, and stencil counts five. Stencil therefore can report a larger GFLOP/s value without processing more outputs per second. Its cached peak also exceeds the nominal 2 GHz compute estimate, reinforcing that the assumed clock and algorithmic FLOP count are not measurements of executed operations.

Implication for SIMD

The data-type comparison supports the expected float32 advantage, but not a universal twofold improvement. Lane count provides potential parallelism; memory traffic and the structure of each kernel determine how much of that potential becomes measured performance.

Alignment and Tail Handling

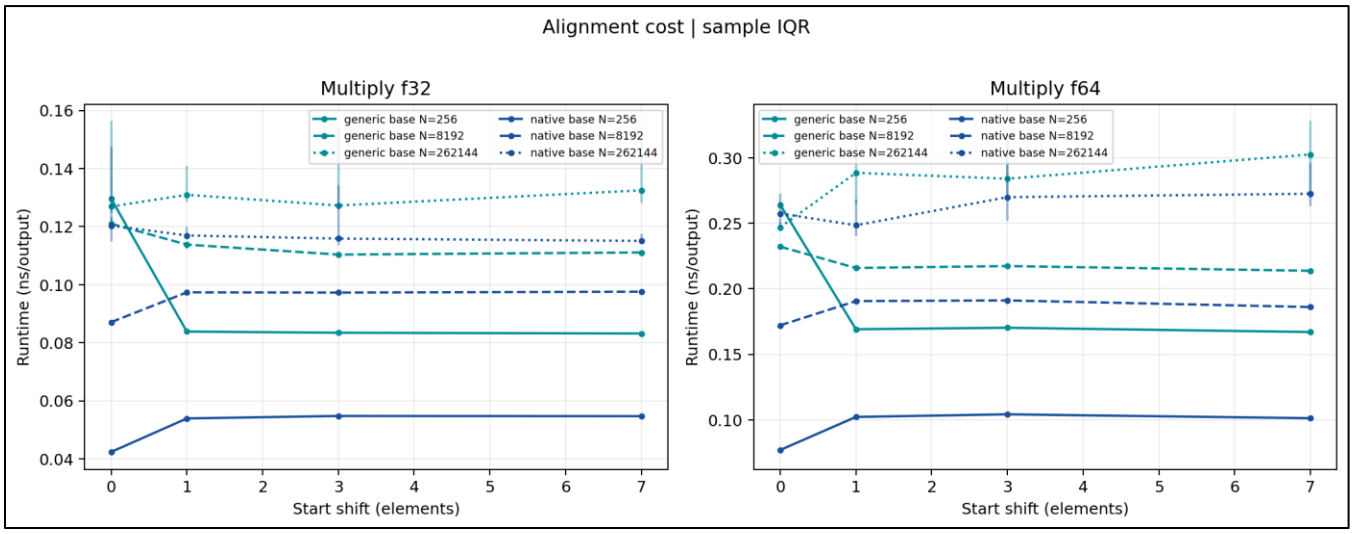


Figure 4. Alignment effects.

At N = 256, shifting the start by one element increases native time per output by 27.2 percent for float32 and 32.8 percent for float64. Both paths remain vectorized, but shifted paths use unaligned memory forms. Larger configurations show small reversals, so alignment provides an advantage in some cases rather than a uniform rule across all sizes.

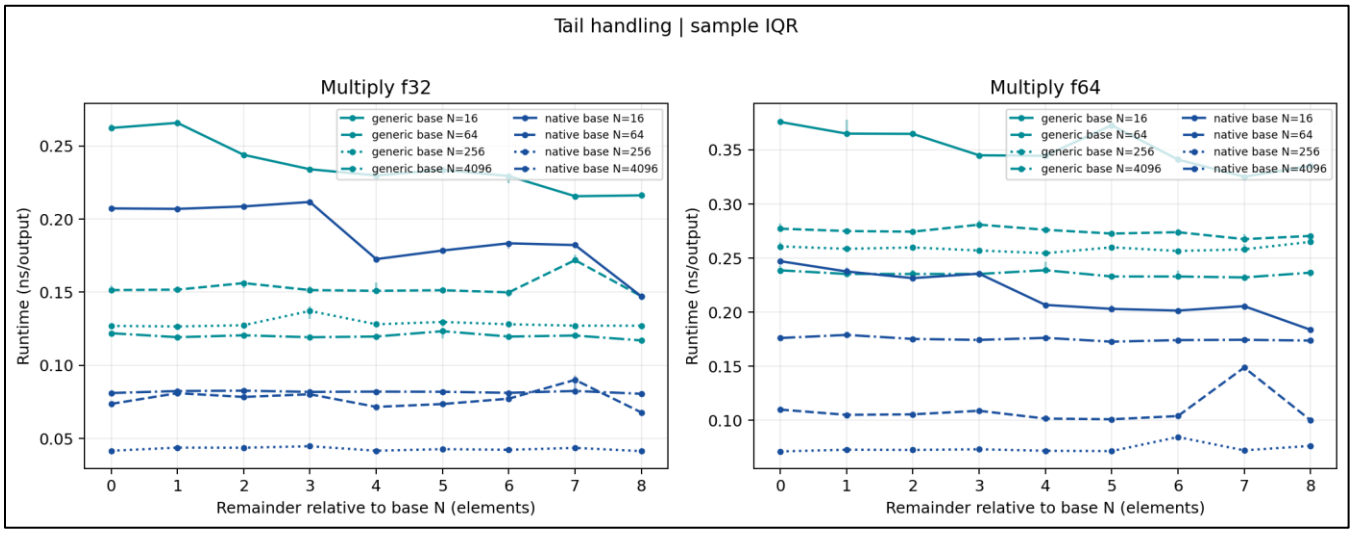


Figure 5. Tail offset effects.

The horizontal axis shows an offset added to a base length, not necessarily the remainder modulo vector width. Adding eight elements can complete whole vectors, and time per output may decrease as fixed overhead is spread over more work. Generic float64 N = 64 and 72 take 17.73 and 19.46 ns per call; both divide evenly by its two-lane width. The original N = 72 spike did not reproduce and cannot be attributed to a leftover scalar element.

Stride and SIMD Speedup

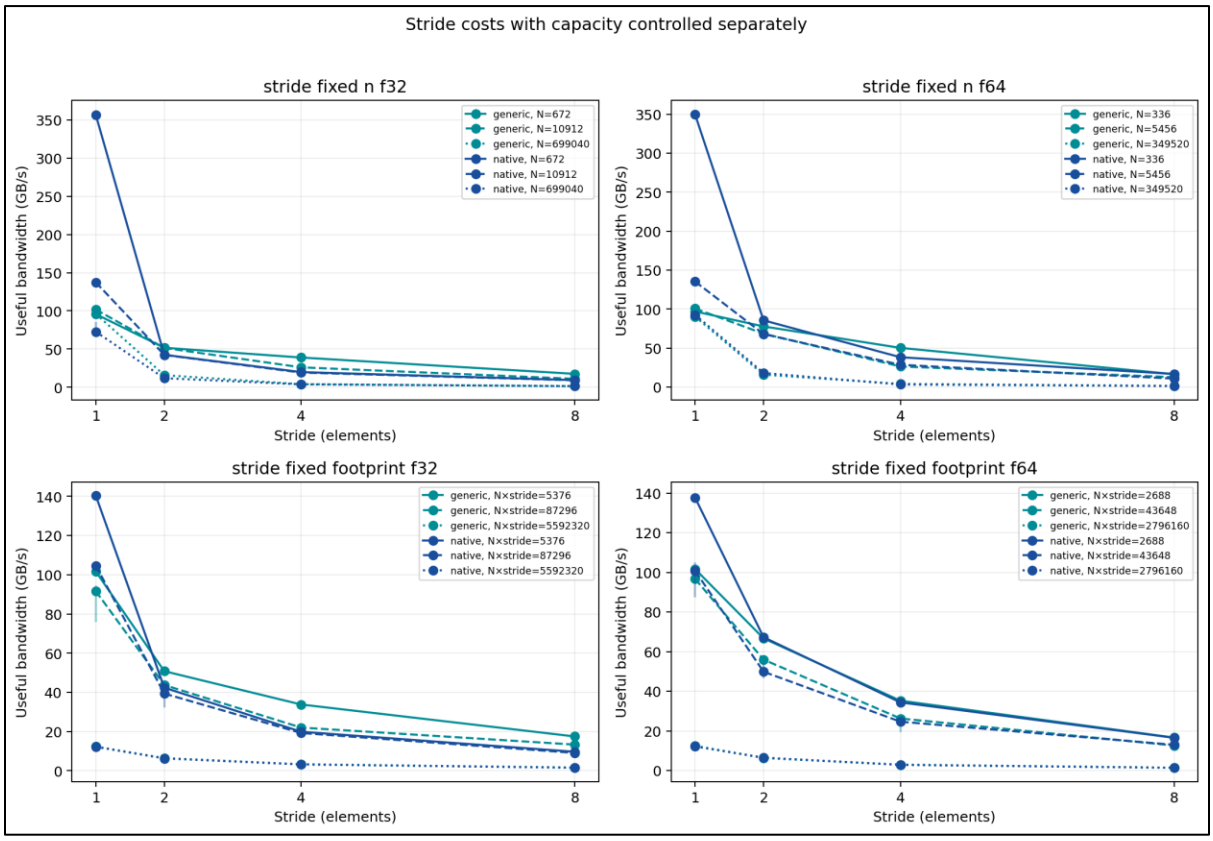


Figure 6. Fixed N and fixed footprint stride controls.

Increasing stride reduces useful throughput in both controls. Near a fixed 63 KiB footprint, native float32 bandwidth falls from 140.35 to 9.63 GB/s between strides one and eight; float64 falls from 137.62 to 16.51 GB/s. Because this penalty remains when footprint is controlled, capacity growth alone does not explain the loss.

Stride reduces the useful data obtained from fetched cache lines and changes the generated instructions. Native stride-8 float32 uses extensive shuffles around packed arithmetic, while native float64 and generic stride-8 paths use scalar arithmetic. No gather instructions appear. Fixed N expands the footprint as stride grows, whereas fixed footprint reduces useful N and changes overhead amortization; the two controls should therefore be interpreted together.

Native Target Comparison

Float32 kernel	Smallest footprint ratio	About 64 MiB ratio
SAXPY	2.046x	1.035x
Dot	1.833x	1.055x
Multiplication	3.203x	0.935x
Stencil	1.990x	1.067x

These ratios divide generic median runtime by native median runtime, so values above one favor native. Native targeting has a larger advantage for cached arrays, but the difference becomes small at approximately 64 MiB. Multiplication is slightly slower with native targeting at that size. Wider instructions therefore do not guarantee an advantage once data access becomes more restrictive. Separate uncertainty intervals were not calculated for these cross-target ratios.

Roofline Interpretation

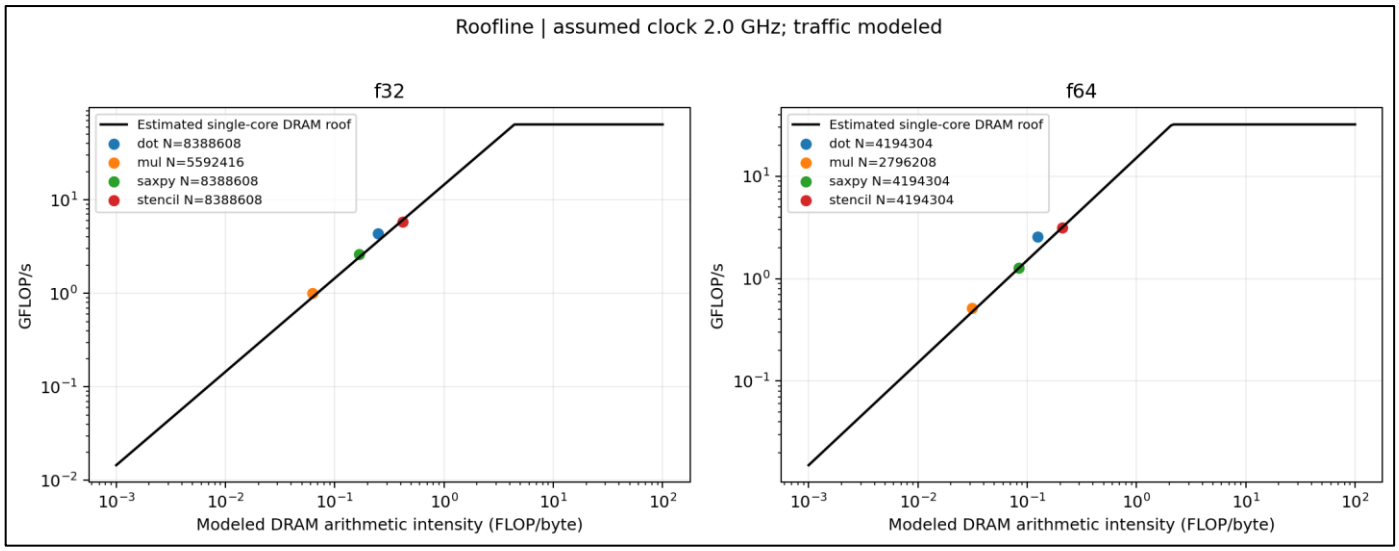


Figure 7. Calibrated roofline estimate.

The roofline compares measured throughput with P = min(Ppeak, B x I), where I is algorithmic FLOPs divided by modeled traffic. Temporal-scale calibration gives B = 14.528 GB/s for float32 and 15.100 GB/s for float64. Assuming 2 GHz, two FMA units, and 256-bit vectors gives nominal compute ceilings of 64 and 32 GFLOP/s. These are model assumptions, and the tested kernels do not use FMA [5].

For float32 multiplication at N = 5592416, F = 5592416 FLOPs and Q = 89478656 modeled bytes give I = 0.0625 FLOP/byte. The estimated bound is 0.9080 GFLOP/s, compared with 1.0022 measured, or 1.104x the estimate. Largest-size dot reaches 1.199x its float32 estimate and 1.360x its float64 estimate.

These apparent violations show that the temporal-scale calibration and traffic assumptions do not provide a universal ceiling. Read-only dot, in-place SAXPY, and output-writing kernels have different access patterns, and their physical memory traffic was not measured. The model remains useful for discussing possible constraints, but its fractions should not be presented as verified hardware utilization. Changing the assumed clock adjusts the compute ceiling without correcting an underestimated bandwidth slope.

Ordered Dot as a Negative Example

Strict dot gains little because preserving summation order leaves a dependent scalar accumulation. The compiler multiplies several pairs together, then extracts and adds their products sequentially into one sum. Relaxed dot instead accumulates in vector lanes before combining them. Native strict-dot speedups are 1.018x and 1.022x for float32, and 1.017x and 0.988x for float64, at N = 4096 and 262144. This demonstrates limited acceleration despite partial SIMD, rather than absence of vectorization.

Measured Cycles per Element

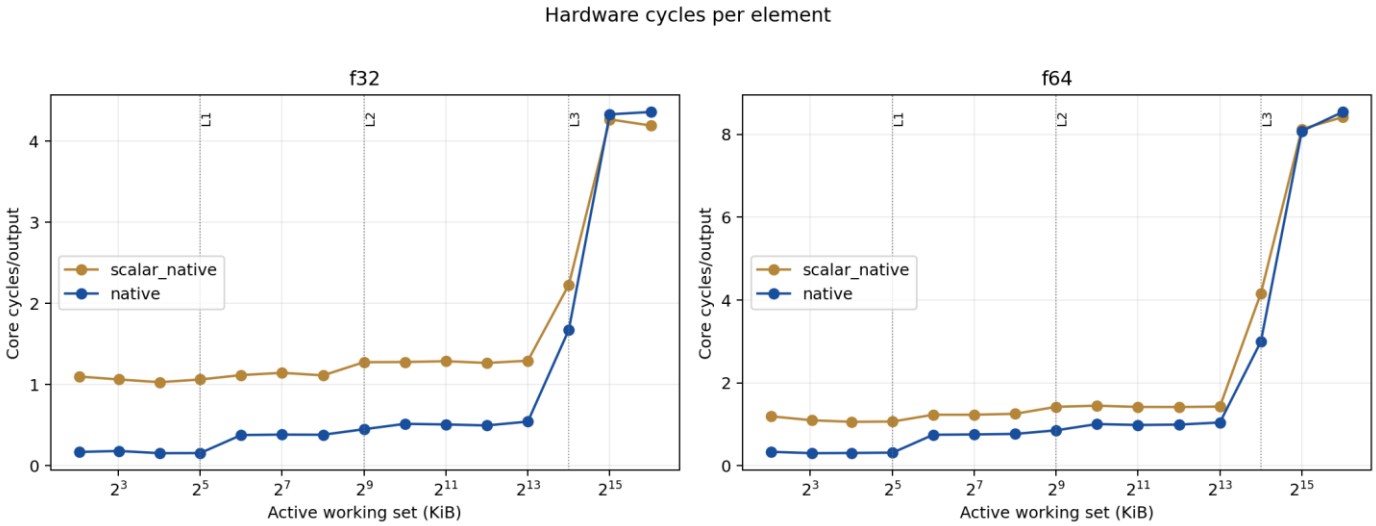


Figure 8. Multiplication core cycles per output.

Case	Native cycles/output	Scalar native cycles/output
Float32 smallest footprint	0.165	1.096
Float64 smallest footprint	0.336	1.193
Float32 about 64 MiB	4.358	4.191
Float64 about 64 MiB	8.539	8.415

For small working sets, native multiplication processes each output using a fraction of the scalar cycle cost. Values below one cycle per output are possible because packed instructions process several values together. As footprints grow, the curves converge, supporting the runtime finding that SIMD provides little advantage for the largest multiplication cases.

Float64 has roughly twice the large-case cycle cost of float32, consistent with twice the data moved per output. These are the benchmark's core-cycle counts, not TSC reference ticks or runtime multiplied by the nominal clock. Core-cycle values are present in every repeat sample. The separate command-line perf probe failed because the executable was absent, while the benchmark used perf_event_open independently. WSL counter exposure and scheduling still limit interpretation [6].

Validation and Reproducibility

Both full runs contain 7560 samples, with no recorded numerical error exceeding its validation bound. Summary medians agree with raw batch times divided by iteration counts. Checks cover short lengths, vector boundaries, shifts, stride holes, unchanged inputs, guard regions, stencil halos, and alternating-sign reductions. Inputs are deterministic finite normal binary fractions; exceptional IEEE values are outside the study.

Median configuration CV falls from 7.51 to 1.69 percent in the repeat. The original large float32 stencil speedup of 0.499x becomes 1.033x (0.985 to 1.192), and the generic float64 N = 72 spike disappears. Kernel objects are byte-for-byte identical. The original notes list OperaGX and Discord, while repeat notes record no background activity. Both runs used Best Power Efficiency, so this comparison does not isolate the cause of the improvement.

Conclusion

The experiments show that SIMD improves performance most when numerical operations are independent and contiguous data can be supplied quickly from cache. Native float32 peak speedups ranged from 5.69x to 8.70x, confirming substantial benefits under these conditions. As working sets exceeded cache capacity, multiplication and stencil approached scalar performance, consistent with data movement becoming more restrictive than arithmetic throughput. Strided access further reduced performance through poor cache-line utilization and additional instruction overhead, while strict dot gained little because ordered scalar accumulation remained a dependency bottleneck despite vectorized multiplication. Overall, generating SIMD instructions is only part of achieving acceleration: the realized advantage depends on memory locality, access pattern, data type, and dependencies within the algorithm.

Measurement and Validation Details

Arrays are allocated on 64-byte boundaries, and strides one, two, four, and eight use specialized loops. N always counts logical outputs, so padding, halos, and shifts do not change the reported work. Elementwise tolerances scale with rounding precision. Dot validation uses a conservative forward-error bound based on N, precision, and the sum of absolute products; large float32 bounds can be loose.

SAXPY resets y before each sample, then updates it repeatedly within the batch. Other kernels reuse fixed inputs. This measures steady-state reuse rather than cold first-touch performance. Case and build-block order use random seed 7530, but consecutive samples within a block can still share thermal or scheduling conditions.

Limitations

The measurements reflect single-thread performance under WSL rather than exclusive access to a physical core. Host scheduling, SMT activity, boost frequency, and thermal conditions may vary between blocks. Both runs used Best Power Efficiency, so the results characterize that operating condition. Seven samples and bootstrap intervals quantify within-run variability but do not capture all variation between runs, as shown by the original stencil slowdown and tail spike failing to reproduce. Warm-up and repeated execution emphasize steady-state behavior, while small cases include function-call and dispatch overhead. Operating frequency and physical DRAM traffic were not measured directly, so the roofline is a calibrated estimate rather than a verified hardware ceiling.

AI Assistance

ChatGPT generated the code as well as supplemental scripts for plotting and execution, and aided in the interpretation of the data, and revision of the report language. The results come from the laptop's outputs when running the code under WSL.

References

[1] AMD. Ryzen 5 PRO 7530U specifications. amd.com/en/support/downloads/drivers.html/processors/ryzen-pro/ryzen-pro-7000-series/amd-ryzen-5-pro-7530u.html. Accessed 12 October 2026.

[2] GCC. Optimize Options. gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html. Accessed 12 October 2026.

[3] GCC. Developer Options. gcc.gnu.org/onlinedocs/gcc/Developer-Options.html. Accessed 12 October 2026.

[4] Microsoft. Advanced settings configuration in WSL. learn.microsoft.com/en-us/windows/wsl/wsl-config. Accessed 12 October 2026.

[5] Williams S, Waterman A, Patterson D. Roofline: an insightful visual performance model for multicore architectures. Communications of the ACM 52(4), 2009, 65-76. doi.org/10.1145/1498765.1498785.

[6] Linux kernel. perf stat documentation. git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/tools/perf/Documentation/perf-stat.txt.